

Tutorial de Instalação do g++ e do OpenCV no Windows

Curso de Visão Computacional

Autor: Fábio Cappabianco

Este tutorial apresenta um caminho simples para instalar um ambiente de desenvolvimento em C++ com OpenCV no Windows usando MSYS2, MinGW-w64 e g++.

Recomendação: para quem vai usar **g++** no Windows, a rota mais prática para aulas iniciais costuma ser **MSYS2 + UCRT64 + MinGW-w64**, porque o compilador, o gerenciador de pacotes e o OpenCV podem ser instalados de forma integrada.

1. Visão geral

Neste guia, o objetivo é instalar três componentes principais: o ambiente MSYS2, o compilador g++ da cadeia MinGW-w64 e a biblioteca OpenCV. Ao final, você terá um programa de teste compilando e executando no Windows.

2. O que será instalado

- MSYS2: distribuição que fornece terminal, gerenciador de pacotes e diferentes ambientes de toolchain.
- MinGW-w64 com g++: compilador C/C++ nativo para Windows.
- OpenCV: biblioteca de visão computacional para leitura, exibição e processamento de imagens.

Resumo do caminho adotado

Componente	Papel	Como será instalado
MSYS2	Ambiente e pacotes	Instalador oficial
g++	Compilação C++	Pacote mingw-w64-ucrt-x86_64-gcc
OpenCV	Biblioteca de visão computacional	Pacote mingw-w64-ucrt-x86_64-opencv

3. Pré-requisitos

- Windows 10 ou Windows 11, preferencialmente 64 bits.
- Conexão com a internet para baixar os pacotes.
- Permissão para instalar programas.
- Algum editor de texto ou IDE. Para começar, até o Visual Studio Code ou um editor simples já resolve.

4. Instalação do MSYS2

A instalação começa pelo MSYS2. Baixe o instalador no site oficial e conclua a instalação com as opções padrão. Ao terminar, abra o terminal chamado UCRT64, que é o ambiente recomendado neste tutorial.

Passos sugeridos

- Acesse o site oficial do MSYS2 e faça o download do instalador.
- Execute o instalador e mantenha o diretório padrão, salvo necessidade específica.
- Ao concluir, abra o atalho do terminal UCRT64.

5. Atualização inicial do sistema

Logo após a instalação, atualize o MSYS2. Esse passo é importante porque o projeto trabalha com atualização completa do sistema.

```
pacman -Syu
```

Se o terminal pedir para ser fechado e aberto novamente, faça isso. Depois, rode o comando de atualização de novo até não haver mais atualizações pendentes.

```
pacman -Syu
```

6. Instalação do compilador g++

No terminal UCRT64, instale o compilador C++ com o pacote do GCC para esse ambiente.

```
pacman -S mingw-w64-ucrt-x86_64-gcc
```

Verificação

Depois, confirme se o compilador foi encontrado corretamente:

```
g++ --version
```

7. Instalação do OpenCV

Com o compilador pronto, instale o OpenCV pelo gerenciador de pacotes do próprio MSYS2.

```
pacman -S mingw-w64-ucrt-x86_64-opencv
```

Teste rápido de descoberta do pacote

Em muitos cenários, o pkg-config facilita a obtenção automática dos parâmetros de compilação e linkedição.

```
pkg-config --modversion opencv4
pkg-config --cflags --libs opencv4
```

8. Programa mínimo de teste

Crie um arquivo chamado main.cpp com o código abaixo. Esse programa apenas imprime a versão do OpenCV instalada.

```
#include <iostream>
#include <opencv2/core.hpp>

int main() {
    std::cout << "OpenCV version: " << CV_VERSION << std::endl;
    return 0;
}
```

Compilação do teste

No mesmo diretório do arquivo, compile com g++ usando os parâmetros fornecidos pelo pkg-config.

```
g++ main.cpp -o teste_opencv `pkg-config --cflags --libs opencv4`
```

Em alguns terminais, pode ser mais confortável usar a substituição com \$() em vez de crases:

```
g++ main.cpp -o teste_opencv $(pkg-config --cflags --libs opencv4)
```

Execução

```
./teste_opencv
```

9. Exemplo com leitura de imagem

Se quiser fazer um teste mais próximo do uso em sala, você pode ler uma imagem e mostrar seu tamanho.

```
#include <iostream>
#include <opencv2/opencv.hpp>

int main() {
    cv::Mat img = cv::imread("imagem.jpg");
    if (img.empty()) {
        std::cerr << "Erro ao abrir a imagem." << std::endl;
        return 1;
    }
}
```

```
std::cout << "Largura: " << img.cols << std::endl;
std::cout << "Altura: " << img.rows << std::endl;
return 0;
}
```

```
g++ main.cpp -o teste_img $(pkg-config --cflags --libs opencv4)
```

10. Estrutura de pastas sugerida

Para os primeiros exercícios, uma estrutura simples costuma ser suficiente:

```
meu_projeto/
|- main.cpp
|- imagem.jpg
```

11. Erros comuns e como resolver

Problema	Possível causa	Solução
g++ não é reconhecido	Terminal errado ou pacote não instalado	Abra o terminal UCRT64 e confira a instalação do pacote gcc.
pkg-config não encontra opencv4	OpenCV não instalado no mesmo ambiente	Reinstale o pacote mingw-w64-ucrt-x86_64-opencv no UCRT64.
Erro ao abrir imagem	Arquivo inexistente ou caminho incorreto	Confirme se imagem.jpg está na mesma pasta do executável ou informe o caminho correto.
Mistura de ambientes	Uso de MINGW64, UCRT64 ou CMD de forma inconsistente	Mantenha instalação, compilação e teste sempre no mesmo ambiente.

12. Quando usar CMake

Para exercícios iniciais, compilar com g++ e pkg-config já é suficiente. Quando o projeto crescer, fizer uso de vários arquivos ou precisar de uma organização melhor, vale migrar para CMake.

13. Alternativa com Visual Studio

O OpenCV também documenta uma rota de instalação no Windows baseada em Visual Studio e CMake. Essa alternativa é útil em projetos maiores ou quando a disciplina adota MSVC. Como o

foco deste material é trabalhar com g++, a rota principal deste tutorial permanece MSYS2 + MinGW-w64.

14. Checklist final

- MSYS2 instalado.
- Sistema atualizado com pacman -Syu.
- g++ respondendo ao comando g++ --version.
- OpenCV respondendo ao comando pkg-config --modversion opencv4.
- Programa de teste compilado e executado.

15. Referências técnicas

Este tutorial foi elaborado a partir da documentação oficial do OpenCV e do MSYS2, especialmente as páginas de instalação no Windows, instalação geral do OpenCV, atualizações do MSYS2 e uso de CMake/MSYS2.